Python Programming Fundamentals

Dictionaries

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1.1 Key-value pairs

A `dict` stores data as **key → value** pairs:

- Keys must be **unique** and **immutable** (strings, ints, tuples)
- Values can be **anything**
- Lookup by key is O(1) — extremely fast

```
student = {
    "name": "Alice",
    "age":  20,
    "grade": "A",
}

# Access by key
print(student["name"])    # Alice
print(student["grade"])   # A
```

Outline

2. Creating Dictionaries

```
# Literal syntax
colours = {"red": "#FF0000", "green": "#00FF00"}

# dict() constructor
prices = dict(apple=1.99, bread=2.50, milk=1.20)

# From two lists
keys   = ["a", "b", "c"]
values = [1, 2, 3]
combo  = dict(zip(keys, values))   # {'a': 1, 'b': 2, 'c': 3}

# Empty dict
registry = {}
```

Outline

3. Accessing and Updating

```python
student = {"name": "Alice", "age": 20}

# Access (raises KeyError if missing)
print(student["name"])   # Alice

# Safe access with .get()
print(student.get("grade", "N/A"))   # N/A (default)

# Update existing key
student["age"] = 21

# Add new key
student["email"] = "alice@example.com"

# Delete key
```

3. Accessing and Updating

```
del student["age"]
print(student)   # {'name': 'Alice', 'email': 'alice@example.com'}
```

Outline

4. Keys, Values, Items

```python
grades = {"Alice": "A", "Bob": "B", "Carol": "A+"}

print(grades.keys())    # dict_keys(['Alice', 'Bob', 'Carol'])
print(grades.values())  # dict_values(['A', 'B', 'A+'])
print(grades.items())   # dict_items([('Alice', 'A'), ...])

# Iterating
for name, grade in grades.items():
    print(f"{name}: {grade}")

# Check if key exists
if "Dave" in grades:
    print(grades["Dave"])
else:
    print("Dave not found")
```

Outline

5. Practical Examples

```
# Word frequency counter
text = "the cat sat on the mat the cat"
freq = {}
for word in text.split():
    freq[word] = freq.get(word, 0) + 1
print(freq)
# {'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1}

# Student registry
registry = {}
registry["alice@example.com"] = {"name": "Alice", "grade": "A"}
registry["bob@example.com"]   = {"name": "Bob",   "grade": "B"}

# Look up
student = registry.get("alice@example.com")
```

5. Practical Examples

```
if student:
    print(student["name"])
```

Outline

6. Dict Comprehensions

```
products = shop.get_all_products()

# {name: price}
price_map = {p.get_name(): p.get_price() for p in products}
# {'Apple': 1.99, 'Bread': 2.50, ...}

# {id: product}
id_map = {p.get_product_id(): p for p in products}
# {101: <Product>, 102: <Product>, ...}

# Filtered: only products over €2
expensive = {p.get_name(): p.get_price()
             for p in products if p.get_price() > 2.0}

# Invert a dict
grades = {"Alice": "A", "Bob": "B"}
```

6. Dict Comprehensions

```
inverted = {v: k for k, v in grades.items()}
# {'A': 'Alice', 'B': 'Bob'}
```

Outline

7. Nested Dicts and defaultdict

```
# Nested dict
contacts = {
    "Alice": {"phone": "555-1234", "email": "a@x.com"},
    "Bob":   {"phone": "555-5678", "email": "b@x.com"},
}
print(contacts["Alice"]["email"])   # a@x.com

# defaultdict — auto-creates missing keys
from collections import defaultdict
word_groups = defaultdict(list)
words = ["apple", "avocado", "banana", "blueberry", "cherry"]
for w in words:
    word_groups[w[0]].append(w)
print(dict(word_groups))
# {'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], ...}
```

Thanks for Watching - Any questions?